Coding Interview Preparation – Key Problems, Solutions & Explanations

Rohit Jangir

1. Maximum Subarray (Kadane's Algorithm)

Problem Explanation

Given an integer array `nums`, find the maximum possible sum of any contiguous subarray. A contiguous subarray contains elements that appear consecutively in the original array.

Example:

$$[-2, 1, -3, 4, -1, 2, 1, -5, 4] \rightarrow \text{Maximum sum is 6 from } [4, -1, 2, 1]$$

Constraints require an efficient approach better than $O(n^2)$, thus we use $O(n)$.

Java Solution

```
public int maxSubArray(int[] nums) {
    int current = nums[0];
    int best = nums[0];

    for (int i = 1; i < nums.length; i++) {
        current = Math.max(nums[i], current + nums[i]);
        best = Math.max(best, current);
    }
    return best;
}
```

Explanation

At each index, we decide whether to:

- Start a new subarray from the current element, or
- Extend the previous subarray.

Thus:

$$current = \max(nums[i], current + nums[i])$$

$$best = \max(best, current)$$

Time complexity: $O(n)$
Space complexity: $O(1)$

2. Longest Substring Without Repeating Characters

Problem Explanation

Given string `s`, return the length of the longest substring without repeating characters.

Example:

$$s = \text{"abcabcbb"} \rightarrow 3 \text{ from "abc"}$$

Java Solution

```
public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> lastIndex = new HashMap<>();
    int left = 0, maxLen = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (lastIndex.containsKey(c) && lastIndex.get(c) >= left) {
            left = lastIndex.get(c) + 1;
        }
        lastIndex.put(c, right);
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Explanation

We maintain a sliding window $[left, right]$ with no duplicate characters. When a duplicate appears, shift `left` forward.

Time: $O(n)$

Space: $O(k)$ where k is charset size.

3. Subarray Sum Equals K

Problem Explanation

Given array `nums` and value `k`, count total number of contiguous subarrays with sum equal to `k`.

$$nums = [1, 1, 1], k = 2 \rightarrow 2 \text{ subarrays}$$

Java Solution

```
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1);
```

```
    int sum = 0, count = 0;
    for (int n : nums) {
        sum += n;
        if (prefixCount.containsKey(sum - k)) {
            count += prefixCount.get(sum - k);
        }
        prefixCount.put(sum, prefixCount.getOrDefault(sum, 0) + 1);
    }
    return count;
}
```

Explanation

Let prefix sum = sum from index 0 to current. A subarray from l to r has sum k when:

$$prefix[r] - prefix[l-1] = k$$

$$prefix[l-1] = prefix[r] - k$$

We count how many times $(sum - k)$ has appeared.

Time: $O(n)$

Space: $O(n)$

4. Climbing Stairs (Dynamic Programming)

Problem Explanation

There are n stairs. You may climb 1 or 2 steps at a time. Return number of distinct ways to reach the top.

Example: $n = 3 \Rightarrow 3$ ways.

Java Solution

```
public int climbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) {
        int c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

Explanation

$$dp[i] = dp[i-1] + dp[i-2]$$

Equivalent to Fibonacci sequence. Use constant space by holding last two states.
Time: $O(n)$
Space: $O(1)$

5. Invert (Mirror) Binary Tree

Problem Explanation

Given a binary tree, swap left and right children recursively.
Example:

$$\begin{array}{ccccc} 4 & \rightarrow & 4 & & \\ 2 & 7 & \rightarrow & 7 & 2 \end{array}$$

Java Solution

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode temp = root.left;
    root.left = root.right;
    root.right = temp;
    invertTree(root.left);
    invertTree(root.right);
    return root;
}
```

Explanation

Swap children at every node, recurse until leaf.
Time: $O(n)$
Space: $O(h)$ for recursion depth.